

The AI Build Cycle

Our operating model for building with AI, from a first idea to a maintained system.
Seven steps, one hard fork by track, grounded in Google's Vibe Coding whitepaper, DORA, GitLab, METR, and Microsoft's own SDL and dogfooding practice.



📍 Why this shape

Google's whitepaper (Osmani, Saboo & Kartakis, May 2026) makes the underlying case: **writing code is no longer the expensive part of engineering — specifying what to build, and verifying what the machine wrote, is**. Two ideas from it shape everything below.

Agent = Model + Harness. The model is roughly 10% of what makes an AI coding agent work; the other 90% is instructions, tools, sandboxes, orchestration, and observability. Most of this document is harness engineering.

Verification rigor is a dial, not a switch. A throwaway spike needs none. A payments change needs a written spec, tests, and a human gate. Every step below states what "enough verification" looks like for that step — match it to what's at stake, and default one notch stricter than feels necessary.

⚠️ **The amplifier principle.** DORA's 2025 report (~5,000 respondents) and Stripe Engineering both land on the same finding independently: AI does not fix a team's problems, it amplifies whatever is already there. A codebase with good tests, clear conventions, and tight CI produces excellent agent output; one without produces the same problems, faster. Fix foundations before scaling AI usage past a pilot.

This is why the model below reshapes the traditional first two SDLC phases (Requirements → Design/Architecture) into a faster loop, rather than just layering AI onto the old linear order: AI has changed the economics of reaching a validated design. Steps 4 onward stay close to standard practice — hardened for AI-assisted work, not reinvented.

🔲 The cycle at a glance

Seven steps are the one spine everyone follows. "Track" is never a parallel pipeline — it only forks the mechanism inside a step where UI, Services/APIs, and Protocol/Spec Pipelines genuinely need different handling.

STEP	TRACK?	OUTCOME PRODUCED
1 · Planning & Requirements	No	Requirements Document
2 · Interactive Prototyping	Hard fork	Approved prototype / proven contract / approved example corpus
3 · Design, Architecture & Tech Docs	Partial	Build-Ready Technical Package
4 · Implementation	No	Implementation Complete — All Tests Passing
5 · Testing & QA	Partial	Test & Quality Report
6 · Dog Fooding	Partial	Dogfood Exit Report
7 · Maintenance	No	Living System Record (<i>continuous</i>)

📦 **Standing rule, not repeated per step:** every outcome needs a named owner who signs off on it. Who that is varies by team and org structure — this guide doesn't prescribe it — but "nobody signed this" is never an acceptable state for any row above.

Planning & Requirements



The old model: a PM writes a doc, hands it to engineering weeks later. Here, requirements-gathering is a fast, direct conversation — but it still ends in something **written down**, because that's what every track's Step 2 works from.

✓ DO

- Interview the AI in reverse: describe the feature, let it ask clarifying questions until it can restate acceptance criteria back to you.
- Write it down — GIVEN/WHEN/THEN acceptance criteria (EARS-style phrasing works well), explicit non-goals, quality targets.
- Decide up front which track this belongs to, and how strict verification needs to be. Say so in the ticket.

✗ DON'T

- Let a plausible-sounding AI-written plan substitute for a domain expert actually reading it. Vague specs get filled with silent assumptions, not flagged gaps.

OUTCOME



Requirements Document

Problem statement, acceptance criteria, non-goals, quality targets, track decision.

Interactive Prototyping



A cyclical loop, not a single pass: **build** → **get real reaction** → **adjust** → **repeat** until the track's outcome is stable — not just "looks done once." This is the one step where the mechanism itself has to change depending on what's being built.

✓ DO

- Get the reaction from whoever actually judges the outcome — the stakeholder, the consuming team, the domain owner — not a proxy.
- Keep cycling until edge cases and disagreements surface, not just the happy path.

✗ DON'T

- Call it done because the first pass worked. The loop's entire job is to surface what a written spec would have missed.

Three tracks, one loop



UI Products

Interactive Prototyping — describe the feature, get a working clickable prototype in minutes, put it in front of the real stakeholder and iterate live.

v0 · bolt.new · Claude Artifacts · Google Stitch · Figma AI

OUTCOME



Approved prototype + cycle log



Services / APIs

Contract-First Prototyping — write the interface contract first, generate a mock server from it, get real consumers integrating against the mock before real implementation exists.

OpenAPI/protobuf + Prism/WireMock · contract tests

OUTCOME



Signed-off contract + proven mock



Protocol / Spec Pipelines

Spec-by-Example — generate concrete input→output example pairs; the domain/protocol owner agrees or corrects specific examples, fast, one at a time.

Diff-generation tooling, reviewed against source-of-truth spec

OUTCOME



Approved golden-example corpus

② Design, Architecture & Technical Documentation



Every source agrees: **architecture stays the most human-owned phase**, even here. What changes is that it's now informed by a validated Step 2 outcome instead of a guess — component/state architecture for UI, data model and service boundaries for APIs, pipeline and state-tracking architecture for protocol work.

The artifact bundle

Architecture doc + diagrams

ADRs

Threat model

Coding standards

AGENTS.md / CLAUDE.md

Model sign-off

UI sign-off

Design sign-off

Testing standards

Smoke test script

NFR / budget doc

Rollout / flighting plan

Phased build plan

✓ DO

- Use AI as a thinking partner — options, trade-offs, a first-pass ADR — but the human owns the actual decision.
- Build the phased build plan the way GitHub Spec Kit and similar tools now do it: **Specify** → **Plan** → **Tasks**, each with its own checkpoint, not one big plan document.
- Write the threat model here, not during testing — Microsoft's own SDL treats it as a design-phase artifact.

✗ DON'T

- Delegate the trade-off itself to the model. It can produce a design that looks defensible and is still wrong for reasons only business context would catch.

OUTCOME



Build-Ready Technical Package

Everything Step 4 needs, nothing it has to guess.

<> Implementation



Built **sub-section by sub-section** against the Step 3 task list. Small commits. Every commit human-reviewed before merge — code review isn't a separate step here, it's built into how implementation happens.

✓ DO

- Maintain a harness file (`CLAUDE.md` / `AGENTS.md` / `copilot-instructions.md`) — stack, conventions, test commands, house rules.
- Red/green: tests written and confirmed failing before implementation, for anything past a prototype.
- Get a fresh-context second review pass on anything non-trivial — a session that didn't write the code, reading the diff cold.
- Watch for the same smells every time: obvious comments, defensive code that doesn't match surrounding patterns, type-system workarounds, invented patterns not used elsewhere in the file.

✗ DON'T

- Rubber-stamp because the diff looks clean. Clean-looking and correct are different properties.
- Ignore it when AI is making you slower on code you already know deeply — METR's 2025 trial found experienced developers 19% slower in exactly that situation, while feeling 20% faster. Take the wheel when that's happening.

OUTCOME



Implementation Complete — All Tests Passing

Every task built, full suite green, no unresolved review comments.

Testing & QA

 Tests stop being something written after code to check it, and become **the mechanism you steer the agent with**. The taxonomy below (grounded in Microsoft's own SDL) is available to every track — which ones carry weight shifts by what's being built.

 **Live Verify — the agent runs the smoke test itself.** Given the detailed smoke-test script from Step 3, the agent drives it directly against the running build and reports pass/fail per step — via Chrome MCP (or equivalent browser automation) for UI, direct calls for services, the golden-example corpus for protocol/spec pipelines. This runs *alongside* human testing, not instead of it.

Unit

Integration

Smoke

Regression

SAST

DAST

Fuzzing

Security regression

Performance / load

Accessibility

✓ DO

- Weight the taxonomy by track — fuzzing matters enormously for protocol parsers, accessibility matters for UI, neither matters much for the other.
- Keep the smoke-test script detailed and current — Live Verify is only as good as the script it follows.
- Scan every AI-assisted change for dependency/secret risk before merge. Up to ~20% of AI-suggested packages have been found not to exist ("slopsquatting" risk).
- Set the verification bar at the eval suite, not the demo.

✗ DON'T

- Accept a mocked integration test without asking why — a mock can pass while the real path is broken.

OUTCOME



Test & Quality Report

Every applicable category run — including agent Live Verify — every critical/high defect fixed or explicitly risk-accepted.

🚩 Dog Fooding



Internal usage before wider release — the release mechanism in this model. What "using it" means shifts by track: employees using a UI app, internal consumer teams integrating against an API before partners do, or a new pipeline run against a subset of real protocols before it's enabled for all.



The amplifier principle shows up here too. Microsoft's own Outlook triage-assistant team reportedly raised accuracy 40% in six months by normalizing inbox policy and sensitivity labels *before* dogfooding the AI feature — not by improving the model. Fix what's underneath before the dogfood ring judges the feature on top of it.

✓ DO

- Define a real exit gate: a telemetry/usage threshold and a feedback-triage process (can be AI-assisted) — not "let some people try it."
- Make the go/no-go decision explicit and owned, not implied by silence.

✗ DON'T

- Skip this step for changes that "feel too small to bother." The exit report is what gives Step 7 a clean starting baseline.

OUTCOME



Dogfood Exit Report

Usage/telemetry vs. threshold, feedback triaged, explicit go/no-go.

Maintenance



The most underrated step in most rollouts. Agents are good at exactly the work that piles up because humans avoid it — and this is the one step whose outcome isn't produced once, it stays true.

✓ DO

- Point AI at legacy modernization, dependency upgrades, and deprecation cleanup — high-tedium work it's well suited for.
- Use AI to document and health-scan unfamiliar code before extending it.
- Use AI to accelerate incident investigation (Ramp reports an 80% cut in investigation time via MCP-connected agents) — keep a human deciding what action to take.

✗ DON'T

- Point autonomous agents at production incident response unsupervised.

OUTCOME



Living System Record (*continuous*)

Docs/ADRs stay current, incidents get postmortems, health scans run on a cadence.

Cross-cutting: security & governance

Not a step — a constraint on all seven.

1. **Write a security-focused harness file.** Input validation rules, "never hardcode secrets," parameterized queries only, a pointer to OWASP ASVS / CWE Top 25. OpenSSF's guide has ready-made templates per language.
2. **Avoid persona prompts** ("act as a security expert") — research shows this tends to make output worse, not better.
3. **The developer is accountable, not the tool.** Whoever's name is on the commit owns the correctness and security of that code.
4. **Have an actual tool policy** — approved assistants, what data they can see, a named owner. GitLab found 49% of teams run 5+ AI tools with no coordinating policy.
5. **Keep an audit trail** for agentic work — what the agent did and why, not just the resulting diff.

Cross-cutting: what to measure

Not lines of code, not percentage of AI-authored commits — both reward volume over judgment.

- **DORA four keys** — deployment frequency, lead time, change failure rate, MTTR — unchanged by AI adoption, still the right signal.
- **Escaped defect rate** for AI-touched vs. human-only changes, tracked for the first few quarters.
- **Perceived vs. actual velocity**, sanity-checked periodically — METR's finding that developers felt faster while measurably running slower means self-report alone isn't reliable here.
- **Team well-being**, not just throughput — DORA's strongest-performing teams score well on both.

Anti-patterns to avoid

Vibe coding in production.

Fine for prototypes and throwaway scripts — not for anything a real user depends on.

Skipping the Step 2 loop.

Jumping to Step 3 because the first pass "looked fine" defeats the entire point of the loop.

The approval rubber-stamp.

Approving without reading defeats the human-in-the-loop gate.

Delegating architecture.

Letting an agent make structural decisions it lacks the business context for.

Mocked tests as a substitute for real verification.

A passing mock isn't evidence the real path works.

Treating Dog Fooding as optional for "small" changes.

It's a real gate, not an informal beta.

Tool sprawl without a policy.

More AI tools isn't more capability without someone coordinating them.

Measuring only speed.

Incident-rate and burnout costs stay invisible until they're already large.

One-page quick reference

- 1 Match verification effort to what's at stake — vibe code a spike, spec-and-test anything customer-facing.
 - 2 Fix engineering foundations before scaling AI usage — it amplifies what's already there.
 - 3 Step 1 always ends in a written Requirements Document — no exceptions.
 - 4 Step 2 is a loop: build, get real reaction, adjust, repeat — mechanism depends on the track.
 - 5 Step 3's Build-Ready Technical Package is what Step 4 works from — never start implementation without it.
 - 6 Small batches, fresh context per task, every commit human-reviewed.
 - 7 Step 5's taxonomy is weighted by track, not applied uniformly.
 - 8 Step 6 is a real gate with an exit report, not an informal beta.
 - 9 Every step's outcome needs a named owner — the guide doesn't say who, but it's never optional.
 - 10 Measure DORA four keys + escaped defects + well-being, not lines-of-code.
-

Research grounding

 **Google — The New SDLC With Vibe Coding**
Osmani, Saboo & Kartakis, May 2026 — the verification spectrum, Agent = Model + Harness, SDLC phase redistribution.

 **DORA / Google Cloud — State of AI-assisted Software Development 2025**
The amplifier principle; seven team archetypes; the AI Capabilities Model.

 **GitLab — 2026 Global DevSecOps Report**
The "AI Paradox" — tool sprawl, governance gap (80% adopted AI faster than governance).

 **METR — Early-2025 AI Impact on Experienced Developer Productivity**
Randomized trial: 19% slower completion despite a perceived 20% speedup, on complex tasks in familiar codebases.

 **OWASP / OpenSSF — Security-Focused Guide for AI Code Assistant Instructions**
Harness-file security templates; dependency hallucination risk.

 **GitHub Spec Kit & the 2026 Spec-Driven Development tooling wave**
The Specify → Plan → Tasks pattern behind Step 3's phased build plan.

 **Microsoft Security Development Lifecycle (SDL)**
The Step 5 testing taxonomy and threat modeling as a design-phase artifact.

 **Microsoft Digital — internal AI dogfooding practice**
The Outlook triage-assistant case study behind Step 6's amplifier callout.

 **Stripe Engineering, Ramp, Faros AI, Thoughtworks, McKinsey (2025)**
"The walls matter more than the model"; MCP-driven incident-response gains; PR-volume-vs-incident-rate trade-offs; AI-native SDLC governance; AI ROI patterns.

The model is not the bottleneck. The team's engineering discipline is.

THIS CYCLE EXISTS TO KEEP THAT DISCIPLINE INTACT WHILE EVERYTHING DOWNSTREAM OF IT GETS FASTER.